Idempotency Keys

how payment APIs never charge twice on retry

Watch the 4-min video

In one line: the client sends one unique `Idempotency-Key` per operation and reuses it on every retry; the server atomically records the key, processes once, stores the response and replays it for duplicates, so a retry can never charge twice.

1 The retry that charges twice

A customer taps **Pay**. Your API charges the card, but the response **times out** on its way back to the phone. The app can't tell whether the charge happened, so it does the sensible thing and **retries**. Without protection, the customer is charged twice.

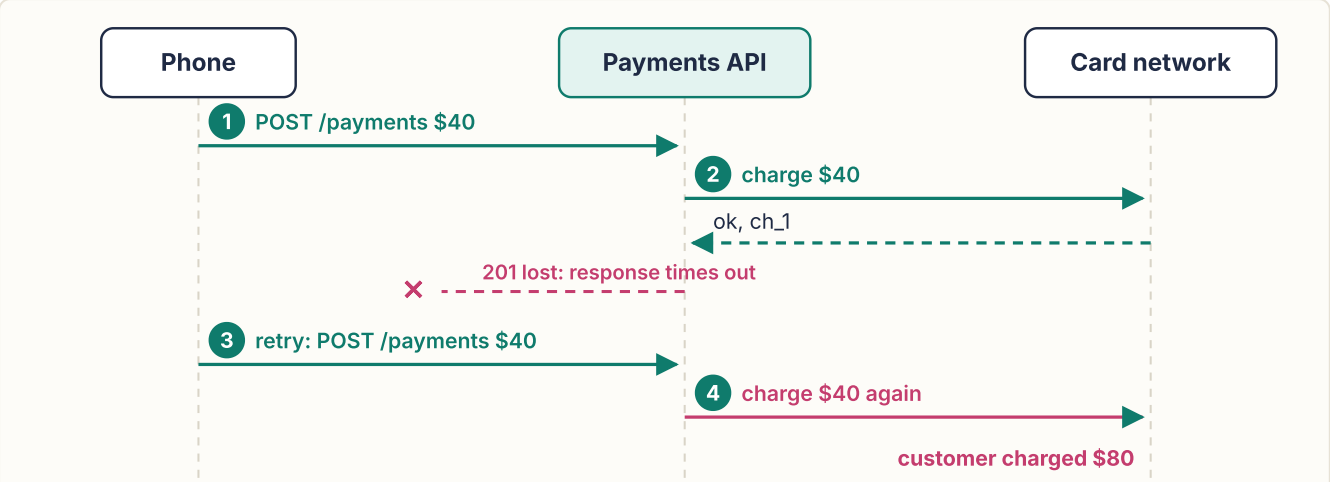


Figure 1. The first request succeeded; only its response was lost. The retry looks like a brand-new payment, so the server charges again.

2 Idempotent operations

An operation is **idempotent** if doing it once or many times leaves the **same result** on the server.

- **GET, PUT, DELETE** are idempotent by definition: putting the same profile twice, or deleting the same order twice, ends in the same state. (The second DELETE may return 404 instead of 204; the state is still the same.)
- **POST** is not: every `POST /payments` can create a new charge.
- Networks give you **at-least-once** delivery: timeouts, client retries, load-balancer retries and queue redelivery mean the same request can arrive more than once. Retries are a fact of life, so the server must make them safe.

3 The idempotency key and the server flow

The client generates a unique ID, such as a **UUID**, **once per logical operation** (one checkout, not one HTTP attempt), and sends it in a header. If it retries, it sends the **same key**.

```
POST /payments
Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7
Content-Type: application/json

{"amount_cents": 4000, "currency": "USD", "card": "tok_visa"}
```

The server stores, with the key, a **fingerprint** of the request: a hash of the method, path and body. Then:

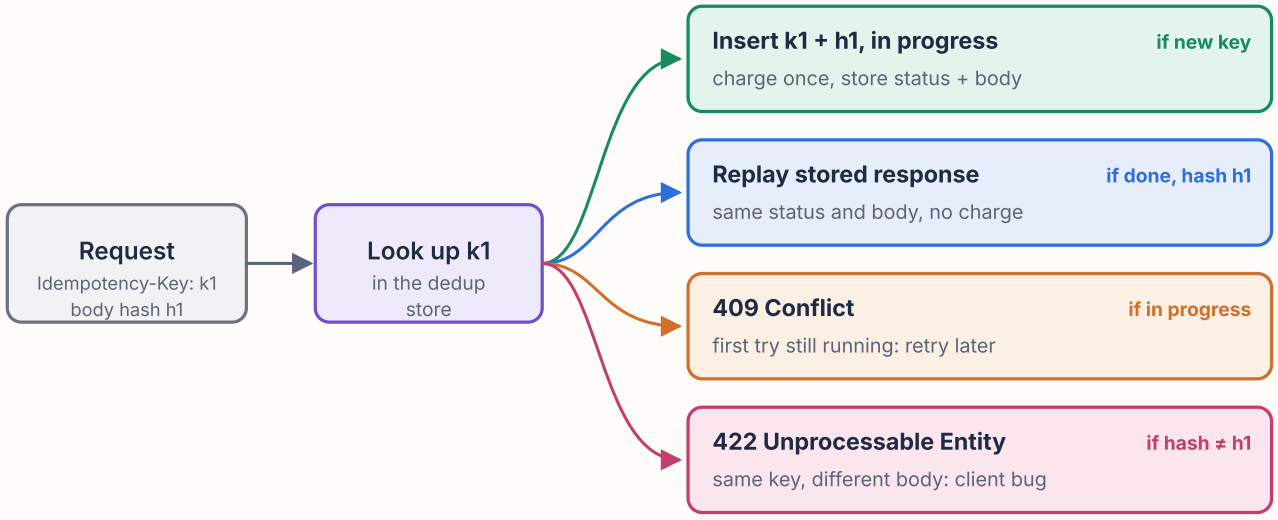
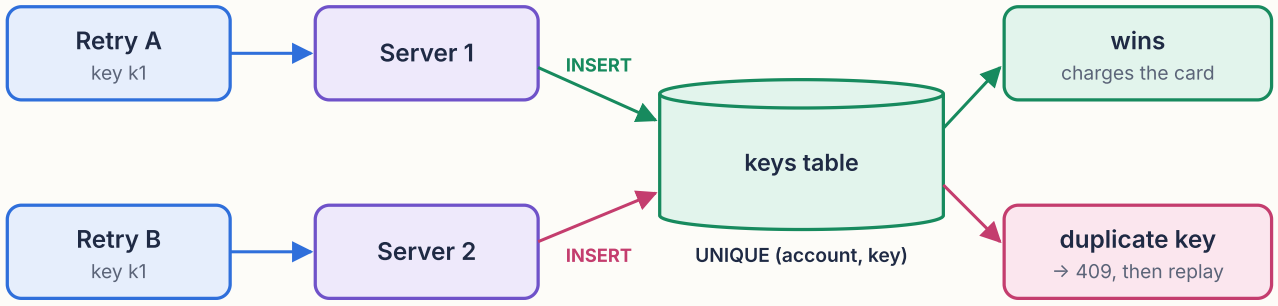


Figure 2. Every request with a key goes through the same lookup. Only a new key leads to a charge; everything else replays, waits or is rejected.

- **New key:** insert it with status *in progress*, process the payment, then save the response status and body with the key and mark it *completed*.
- **Exists and completed:** replay the stored response: same status code, same body (the original `201` with charge `ch_1`).
- **Exists and still in progress:** return `409 Conflict`, so the client retries a little later.
- **Same key, different fingerprint:** that's a client bug (it reused a key for a different payment). Reject with `422` rather than replaying a response that belongs to another request.

4 Concurrent duplicates: make the insert atomic

The tricky case: two retries arrive **at the same moment, on different servers**. If both do "look up, not found, insert", both charge. So the claim must be **atomic**: a **unique constraint** on the key in Postgres, or `SET … NX` in Redis. Only one request wins; the other sees that the key already exists.



Redis equivalent: SET idem:acct42:k1 in_progress NX EX 86400 (only one caller gets OK)

Figure 3. Both servers try to insert the same key. The database lets exactly one succeed; the loser gets a unique-violation and answers 409, and a later retry replays the stored result.

Ideally, **write the key in the same database transaction as the payment record**, so the two can never disagree: either both exist or neither does. If you use Redis for speed, add a recovery rule for keys stuck *in progress* after a crash (for example, expire the claim after a few minutes and check with the processor before charging again).

```
CREATE TABLE idempotency_keys (
  account_id    BIGINT NOT NULL,
  key           TEXT   NOT NULL,
  request_hash  TEXT   NOT NULL,
  status        TEXT   NOT NULL,          -- 'in_progress' | 'completed'
  response_code INT,
  response_body JSONB,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, key)
);
```

5 Where to store keys, and for how long

- **Scope keys per account** (`PRIMARY KEY (account_id, key)`), so two customers can never collide or read each other's responses.
- **Keep them long enough** to cover every realistic retry. Stripe keeps keys for at least **24 hours**, then prunes them.
- **Redis with a TTL** is fast and expires keys for you; a **database table** is durable and can share a transaction with the payment.
- **Pass the key downstream** to the payment processor, so the whole chain is idempotent, not just your API.

WORKED EXAMPLE

Peak **2,000 payments/s**, keys kept **24 h**. Daily volume is lower than peak; say an average of 500/s: 500 × 86,400 ≈ **43 million keys** live at once. At ~1 KB each (key, hash, stored response) that's about **43 GB**: fine for a Postgres table pruned daily by `created_at`, or a Redis cluster with `EX 86400`. Retries are a small share of traffic, so most lookups find nothing and cost one index probe.

COMMON MISTAKES

- Generating a new key on every retry: then the key protects nothing.
- “Check then insert” without a unique constraint: two concurrent retries both charge.
- Ignoring the request fingerprint, so a reused key replays the wrong payment.
- Global keys not scoped per account.
- Storing the key in Redis but the payment in Postgres with no recovery for crashes in between.
- Not sending the key to the downstream processor.

6 Key terms

Idempotent: once or many times, same result

Dedup store: table or cache of seen keys

Atomic claim: unique insert / SET NX: one winner

At-least-once: a message may arrive more than once

Request fingerprint: hash of method, path and body

TTL: how long a key is kept

Idempotency key: client-made unique ID per operation

Replay: return the stored original response

Key scope: keys unique per account

7 Interview questions

Q1. How do you make a payment API idempotent? BASIC
The client sends a unique Idempotency-Key per operation and reuses it on retries. The server atomically inserts the key with a request fingerprint, processes once, stores the response and replays it for duplicates: 409 while in progress, 422 for a different body.

Q2. Which HTTP methods are idempotent? BASIC
GET, HEAD, PUT and DELETE (and OPTIONS). POST is not, and PATCH isn't guaranteed. That's why POST /payments needs an idempotency key.

Q3. Where do you store keys, and for how long? BASIC
In a durable store, ideally in the same transaction as the payment, or Redis with a TTL; scoped per account; for at least the retry window, typically 24 hours (Stripe keeps them at least 24 h).

Q4. Two identical retries hit different servers 5 ms apart. What prevents a double charge? SCENARIO
An atomic claim: a unique constraint on (account_id, key) or Redis SET NX. Exactly one insert succeeds; the other gets a conflict, returns 409, and its next retry replays the stored response.

Q5. A client reuses the same key for a $40 and then a $90 payment. What happens? SCENARIO
The fingerprint (hash of method, path, body) differs, so the server rejects the second with 422 instead of replaying the $40 result or charging $90 under the old key.

Q6. Your server crashes after charging the card but before saving the response. What then? SCENARIO
The key is stuck in progress. If the key and payment are in one transaction, the payment row tells you the outcome. Otherwise pass the key to the processor so a retried charge is deduplicated there, and reconcile stuck keys after a timeout.

Q7. Why not just deduplicate on (card, amount) within a minute? TRICKY
Two real $40 purchases a minute apart are legal, and they'd be blocked; a slow retry after 61 s would still double charge. Only the client knows which attempts are the same operation, so it must say so with a key.

Q8. Should a replayed response come from the cache even if the original was a 500? TRICKY
Store only responses that reflect a final outcome. If the first attempt failed before any side effect, release the key so a retry can run again; replaying a transient 500 forever would block the payment.

Next: SD-C-010 · API Request Validation backendroadmap.com